Unidad I - Aplicaciones Web

1. Proyecto que debes construir

Crea una aplicación llamada **"Sistema de Servicios"**. Puedes ambientarla como clínica, gimnasio, biblioteca, taller, veterinaria o negocio de tu elección. La funcionalidad obligatoria es la misma para todos.

IMPORTANTE: Para la demostración final NO uses Live Server. La página debe abrirse desde Express en http://localhost:3000.

2. Preparación del proyecto

1. Crea una carpeta con el nombre: Apellido_Nombre_Unidad1.
2. Ábrela en Visual Studio Code.
3. Desde la terminal inicializa Node.js e instala Express:

```
npm init -y
npm install express
```

4. Organiza el proyecto exactamente con esta estructura:

```
Apellido_Nombre_Unidad1/
│
├── public/
│   ├── index.html
│   ├── estilos.css
│   └── script.js
│
├── app.js
├── package.json
└── package-lock.json
```

3. Frontend

En index.html y estilos.css construye una interfaz funcional que incluya:

- Encabezado con el nombre de tu sistema.
- Menú con al menos 3 opciones.
- Una sección principal de bienvenida.
- Un contenedor vacío donde se cargarán los servicios desde el backend.
- Un formulario con: nombre, correo, servicio y comentario.
- Un botón para enviar la solicitud.
- Un espacio visible para mostrar mensajes de éxito o error.
- Pie de página con tu nombre.

En CSS aplica obligatoriamente:

- Paleta de colores propia.
- Flexbox o Grid.
- Margin y padding.
- Bordes o tarjetas.
- Botón con efecto hover.

- Distribución ordenada y legible.

4. JavaScript del navegador

En script.js debes programar estas acciones:

- Validar que el nombre y el servicio no estén vacíos.
- Mostrar en la página un mensaje de error cuando falte un dato obligatorio.
- Mostrar en la página un mensaje de éxito cuando el servidor acepte la solicitud.
- Evitar que el formulario recargue la página al enviarse.

5. Backend con Node.js y Express

En app.js configura el servidor para que cumpla TODOS estos requisitos:

- Servir la carpeta public como contenido estático.
- Aceptar datos JSON enviados desde el navegador.
- Ejecutarse en el puerto 3000.
- Crear GET /api/servicios.
- Crear POST /api/solicitudes.
- Crear una respuesta 404 para rutas que no existan.

GET /api/servicios

Debe devolver un arreglo JSON con mínimo 3 servicios. Cada servicio debe tener, como mínimo: id, nombre y descripción.

```
[
  { "id": 1, "nombre": "Servicio 1", "descripcion": "..." },
  { "id": 2, "nombre": "Servicio 2", "descripcion": "..." },
  { "id": 3, "nombre": "Servicio 3", "descripcion": "..." }
]
```

POST /api/solicitudes

Debe recibir los datos enviados por el formulario y responder con estado 201 y un JSON de confirmación. Si faltan datos obligatorios, debe responder con estado 400.

> **NO necesitas base de datos:** En esta práctica basta con recibir, validar y devolver los datos. No agregues MySQL, MongoDB, Angular ni otros frameworks.

6. Investigación obligatoria -

ESTA PARTE NO VIENE RESUELTA: Investiga cómo usar fetch() en JavaScript para hacer una petición GET a /api/servicios, convertir la respuesta a JSON y mostrar los servicios dentro del HTML usando el DOM.

Tu investigación debe terminar en una implementación funcionando. No debes entregar un reporte ni copiar una definición; debes demostrarlo en tu aplicación.

Cuando funcione el GET, utiliza la misma idea de fetch() para que el formulario envíe sus datos a POST /api/solicitudes y muestre la respuesta del servidor en pantalla.

7. Pruebas y demostración.

Antes de llamar al docente, verifica que puedas demostrar, en este orden:

1. Ejecutas node app.js y la terminal muestra que el servidor está activo.

2. Abres http://localhost:3000 y carga tu página con diseño.

3. Los servicios aparecen en la página después de obtenerlos desde /api/servicios.

4. Abres /api/servicios en el navegador y se observa el JSON.

5. Llenas el formulario, lo envías y recibes confirmación sin recargar la página.

6. Dejas un dato obligatorio vacío y tu aplicación muestra un error.

7. Abres una ruta inexistente y el servidor responde 404.

8. Muestras en VS Code dónde están HTML, CSS, JavaScript, GET, POST y la llamada fetch().

9. Entrega

- Demostración funcionando frente al docente al terminar el tiempo.
- Carpeta completa comprimida en ZIP.
- Nombre del archivo: Apellido_Nombre_Unidad1.zip.
- No entregues únicamente capturas de pantalla: el proyecto debe ejecutarse.

10. Evaluación

Criterio	Valor
HTML y CSS funcionales y ordenados	15%
JavaScript y validación	15%
Servidor Node.js + Express funcionando	20%
GET, POST, JSON y códigos de respuesta	20%
Investigación e integración de fetch()	20%
Demostración completa y proyecto organizado	10%

REGLA DE LA PRÁCTICA: Puedes consultar tus apuntes, documentación o una IA para resolver dudas puntuales. Si utilizas código que no puedes explicar ni modificar, la parte correspondiente no se considera completada.

Checklist

□ Mi página abre desde Express.
□ Mi GET devuelve JSON.
□ Mi frontend carga los servicios desde el backend.
□ Mi formulario envía datos al POST.
□ Muestro éxito y error en pantalla.
□ Tengo una respuesta 404.
□ Puedo explicar dónde está cada parte de mi proyecto.